Qassim University جامعة القصيم

Semester 443 - 2023, 1444

Course CS315 Algorithms Analysis and Design

Greedy algorithms

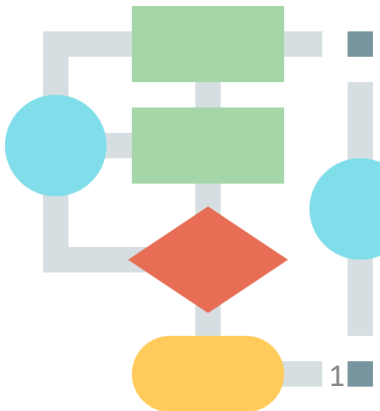
Lecture 5

Prof. Dr Mohammed Al-Hagery

Supervisordr@yahoo.com

Athar alwabel

aa.alwabel@qu.edu.sa



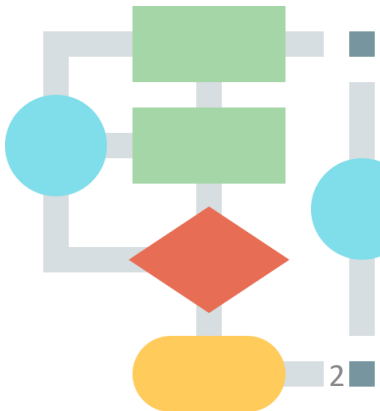
Greedy algorithms

A greedy algorithm is an approach for solving a problem by selecting the best option available at the moment. It doesn't worry whether the current best result will bring the overall optimal result.

The algorithm never reverses the earlier decision even if the choice is wrong. It works in a top-down approach.

Greedy algorithms and it is application

1. **An activity-selection problem .**
2. **Elements of the greedy strategy**
3. **Huffman codes**



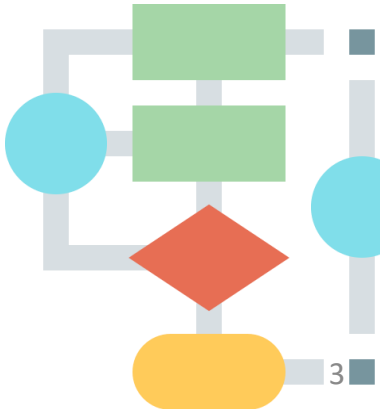
An activity-selection problem

- We are given a set S of n activities S={a1, . . . , an}
- Each activity ai has a start time si and finish time fi, where i-th activity ai = [si, fi) starts at time si and finishes at time fi.

Where 0<=si<=fi<= ∞

- These activities require the same resource (for example, one lecture hall).
- Two activities ai and aj are compatible if the intervals [si, fi) and [sj , fj) do not overlap, i.e they are disjoint.

- The activity-selection problem (ASP) is to select a maximum size subset of mutually compatible activities.



An activity-selection problem

Qassim University جامعة القصيم

Example

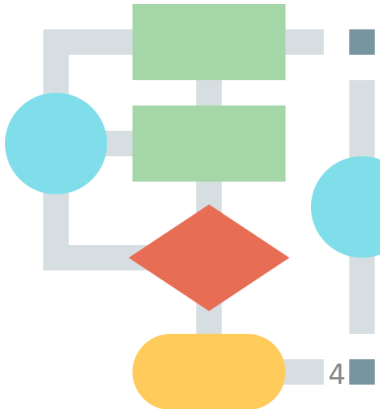
Consider following set of activities,

a	1	2	3	4	5	6	7	8	9	10	11
Si	1	3	0	5	3	5	6	8	8	2	12
Fi	4	5	6	7	9	9	10	11	12	14	16

I is the task, si is start tim e, fi is finish time

The subset {a3, a9, a11}, {a1, a4, a8, a11} (a2, a4, a9, a11}, no overlapping in the intervals.

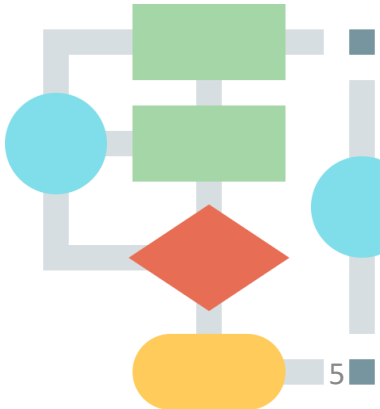
Are Compatible Activities.



An activity-selection problem

- Assume activities sorted in increasing order of finish time fi
- Let c[i,j] be the number of activities in the maximal solution for the subset Sij, i.e. the largest number of compatible activities *(not overlapped)* starting from time fi, finishing at time sj.
- Recursively
- C[I,j]= {max i<k<j C[l,k]+C[k,j]+1 otherwise}
- Then the maximum number of compatible activities is c[0, n + 1].
- Greedy choice. Select an activity with minimum fi. Remove it and incompatible activities. Continue until no more activities left.

- This can be implemented with a **recursive algorithm.**



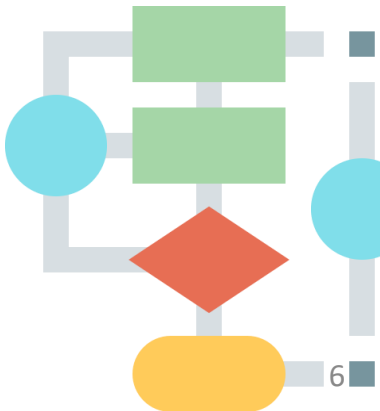
Recursive An activity-selection problem

```
RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)
1  m = k + 1
2  while m ≤ n and s[m] < f[k]        // find the first activity in S_k to finish
3      m = m + 1
4  if m ≤ n
5      return {a_m} ∪ RECURSIVE-ACTIVITY-SELECTOR(s, f, m, n)
6  else return ∅
```

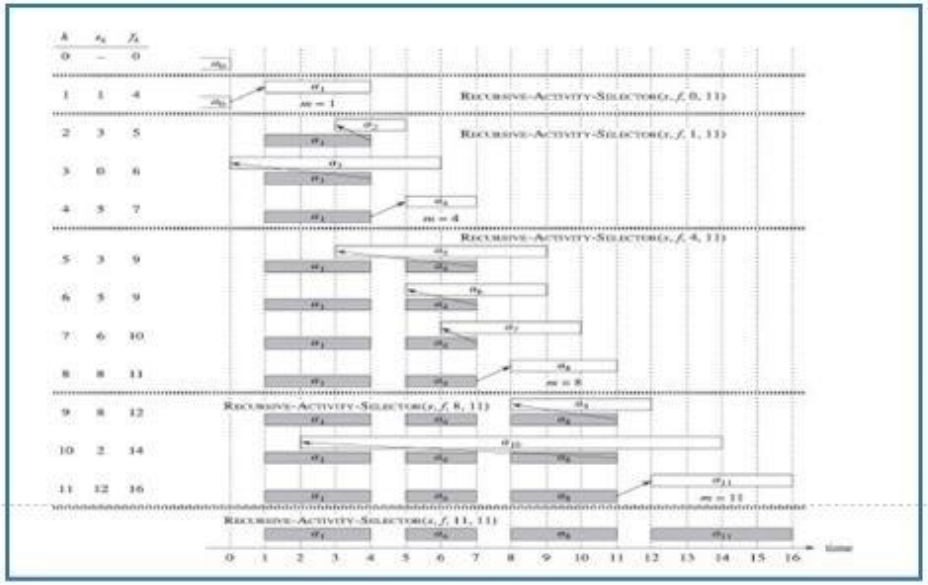
The running time of the Algorithm is ....

Because over all recursive calls, each activity is examined exactly once.

So it takes constant time Θ (n) for n activities.



i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16



First, I will select task 1 which starts at 1 and finishes at 4

1				

Task 2 & 3 are rejected because of their start point is less than the finish time of task 1.

Task 4 was accepted because it starts after the duration of Task 1 ends.

1	4			

Task 5 & 6 & 7 are rejected because of their start point is less than the finish time of task 4.

Task 8 was accepted because it starts after the duration of Task 4 ends.

1	4	8		

Task 9 & 10 are rejected because of their start point is less than the finish time of task 8 .

Task 11 was accepted because it starts after the duration of Task 8 ends.

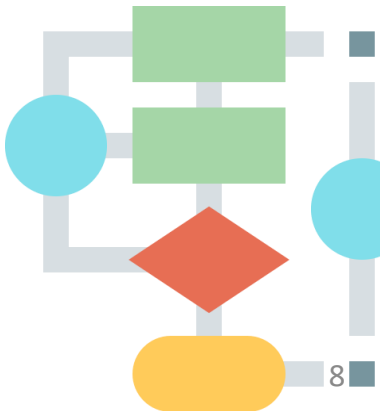
1	4	8	11	

Greedy algorithm of n activity-selection problem

- In this algorithm the activities are sorted according to their finishing time, from the earliest to the latest. Then the activities are greedily selected by going down the list and by picking whatever activity that is compatible with the current selection.
- It collects selected activities into a set A and returns this set when it is done

```
GREEDY-ACTIVITY-SELECTOR(s, f)
1  n = s.length
2  A = {a_1}
3  k = 1
4  for m = 2 to n
5      if s[m] ≥ f[k]
6          A = A ∪ {a_m}
7          k = m
8  return A
```

Like the recursive version, GREEDY-ACTIVITY-SELECTOR schedules a set of n activities in Θ (n) time, assuming that the activities were already sorted initially by their finish times.

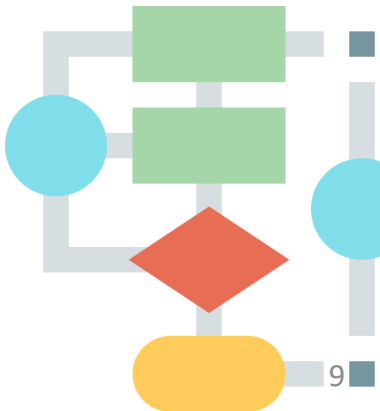


Elements of the greedy strategy

- A greedy algorithm obtains an optimal solution to a problem by making a sequence of choices. At each decision point, the algorithm makes choice that seems best at the moment. This heuristic strategy does not always produce an optimal solution, but as we saw in the activity-selection problem, sometimes it does. This section discusses some of the general properties of greedy methods.

The greedy algorithms design according to the following sequence of steps:

- Cast the optimization problem as one in which we make a choice and are left with one subproblem to solve.
- Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the greedy choice is always safe.
- Demonstrate optimal substructure by showing that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem.



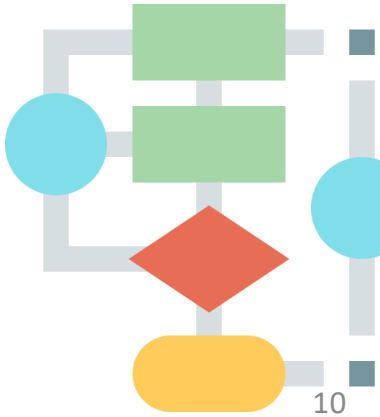
Elements of the greedy strategy

Greedy-choice property

- The first key ingredient is the greedy-choice property: a globally optimal solution can be arrived at by making a locally optimal (greedy) choice. In other words, when we are considering which choice to make, we make the choice that looks best in the current problem, without considering results from sub problems.

Optimal substructure

- A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems. This property is a key in-gredient of assessing the applicability of dynamic programming as well as greedy algorithms.



Elements of the greedy strategy

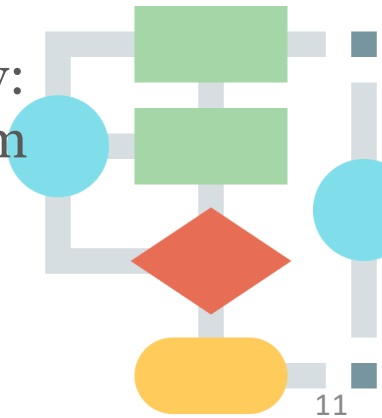
Example

The 0 - 1 knapsack problem:

A thief has a knapsack that holds at most W pounds. Item i : (vi, wi) (v = value, w = weight) thief must choose items to maximize the value stolen and still fit into the knapsack. Each item must be taken or left (0 - 1).

Fractional knapsack problem:

takes parts, as well as wholes
Both the 0 - 1 and fractional problems have the optimal substructure property:
Fractional: vi / wi is the value per pound. Clearly you take as much of the item with the greatest value per pound. This continues until you fill the knapsack.
Optimal (Greedy) algorithm takes O (n lg n), as we must sort on vi / wi = di.



Elements of the greedy strategy

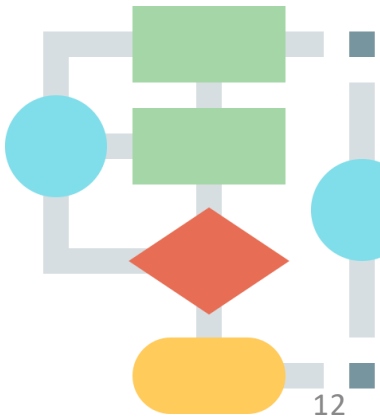
Example

Consider the same strategy for the 0 - 1 problem:
were d is the value density, W = 50 lbs رطل. (maximum knapsack capacity) اكثر شيء ممكن يشال

w1 = 10	v1 = 60	d1.= 6
w2 = 20	v2 = 100	d2.= 5
w3 = 30	v3 = 120	d3 = 4

Greedy approach: Take all of 1, and all of 2: v1+ v2 = **160**,
optimal solution is to take all of 2 and 3: v2 + v3= **220,**
other solution is to take all of 1 and 3 v1+ v3 = 180.

All below **50** lbs .



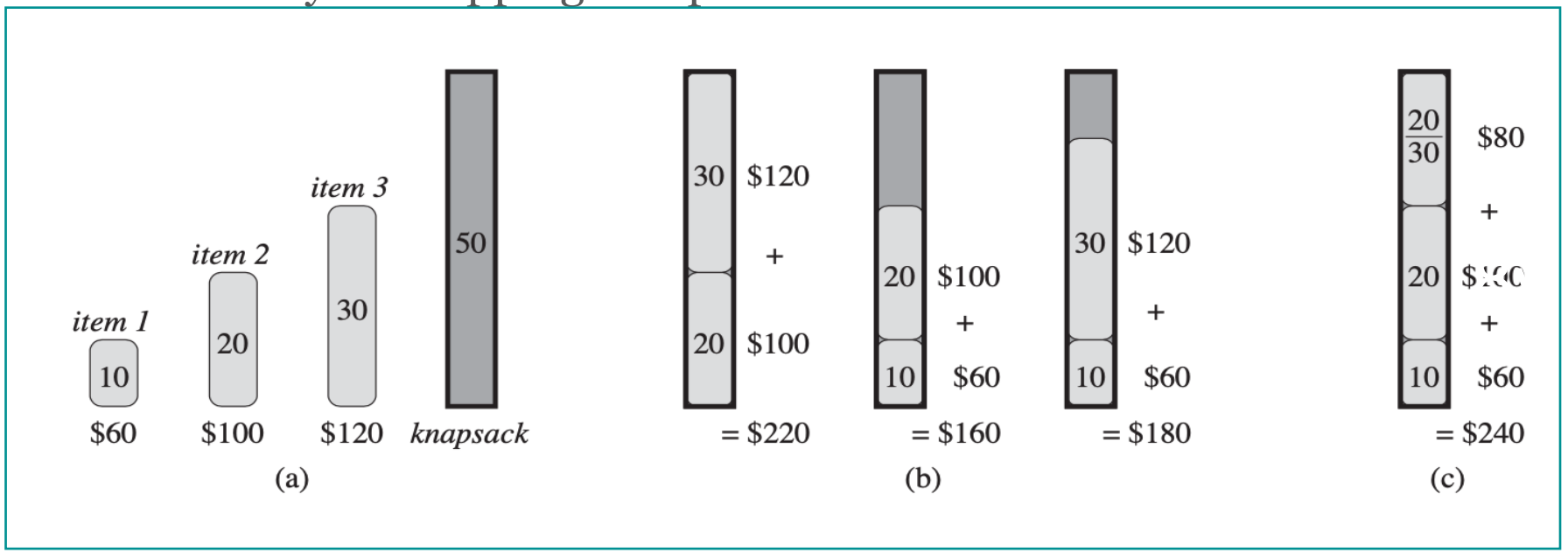
Elements of the greedy strategy

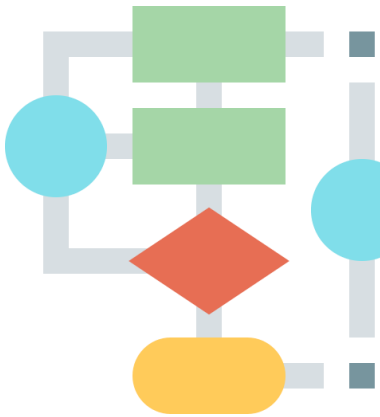
Example

When solving the 0 - 1 knapsack problem, empty space lowers the effective d of the load.
Thus each time an item is chosen for inclusion we must consider both

- included
- excluded

These are clearly overlapping sub-problems for different i's and so best solved by DP!





Huffman codes

- **Our objective is to develop a code that represents a given text as compactly as possible.**
- **A standard encoding is ASCII, which represents every character using 8 bits**

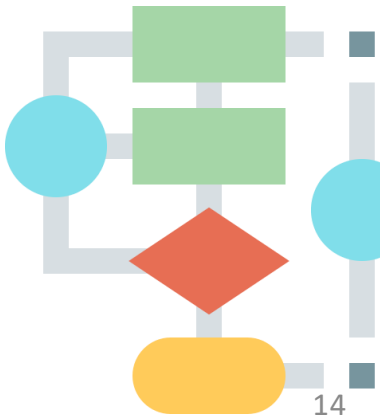
Letter	ASCII Code	Binary Form
A	65	01000001
B	66	01000010
C	67	01000011
D	68	01000100
E	69	01000101

Message: (BCCABBDDAECCBBAEDDCC)

Length-20

In Electric components the alphabet is sent through ASCII code . The ASCII code letter capital A is 65 and we need 8 bis binary to convert 65.

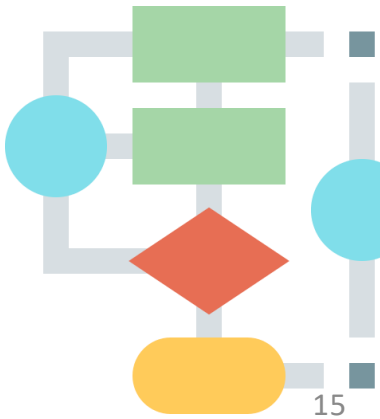
- For 1 Letter We need 8 bits
- For 20 Letters We need 8×20=160 bits



Huffman codes Algorithm

```
HUFFMAN(C)
1  n = |C|
2  Q = C
3  for i = 1 to n − 1
4       allocate a new node z
5       z.left = x = EXTRACT-MIN(Q)
6       z.right = y = EXTRACT-MIN(Q)
7       z.freq = x.freq + y.freq
8       INSERT(Q, z)
9  return EXTRACT-MIN(Q)      // return the root of the tree
```

Time complexity of Huffman Coding is O(n log n) , where n is the number of unique characters.

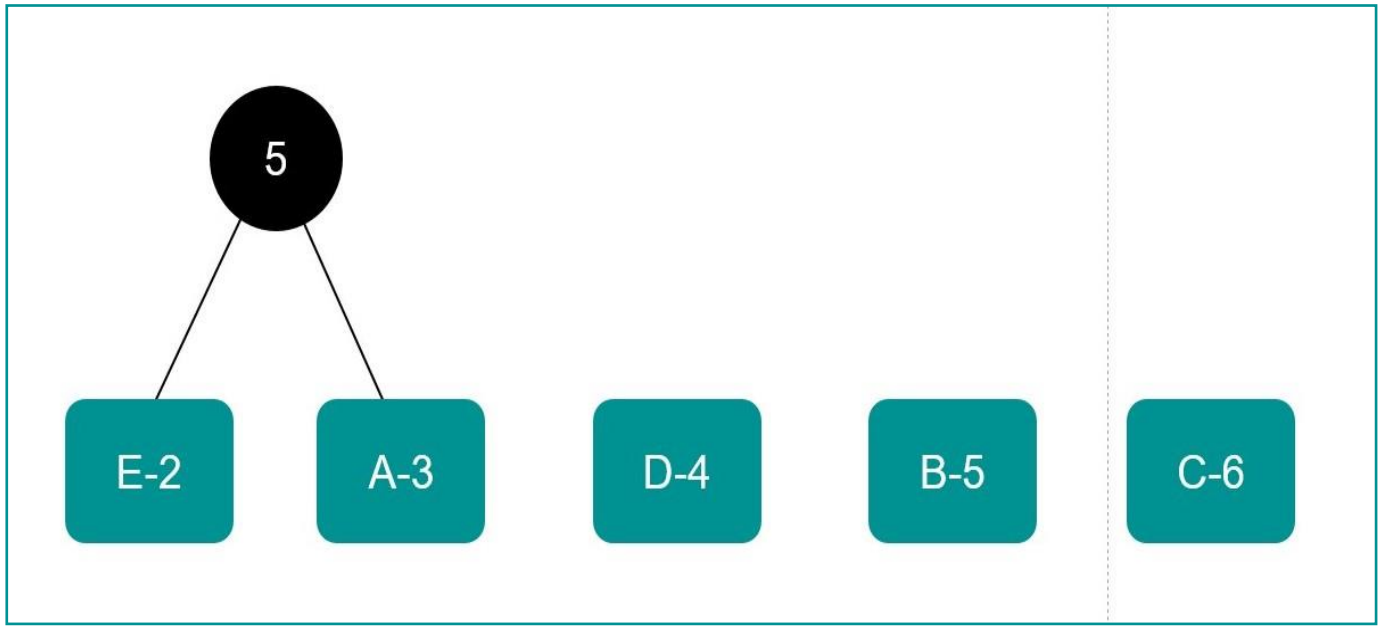


Huffman codes Algorithm

Example .. **Message: (BCCABBDDAECCBBAEDDCC)**

First of all place the counts in increasing order then take minimum and add them now to the root node of letter E and A is 5

التكرارت

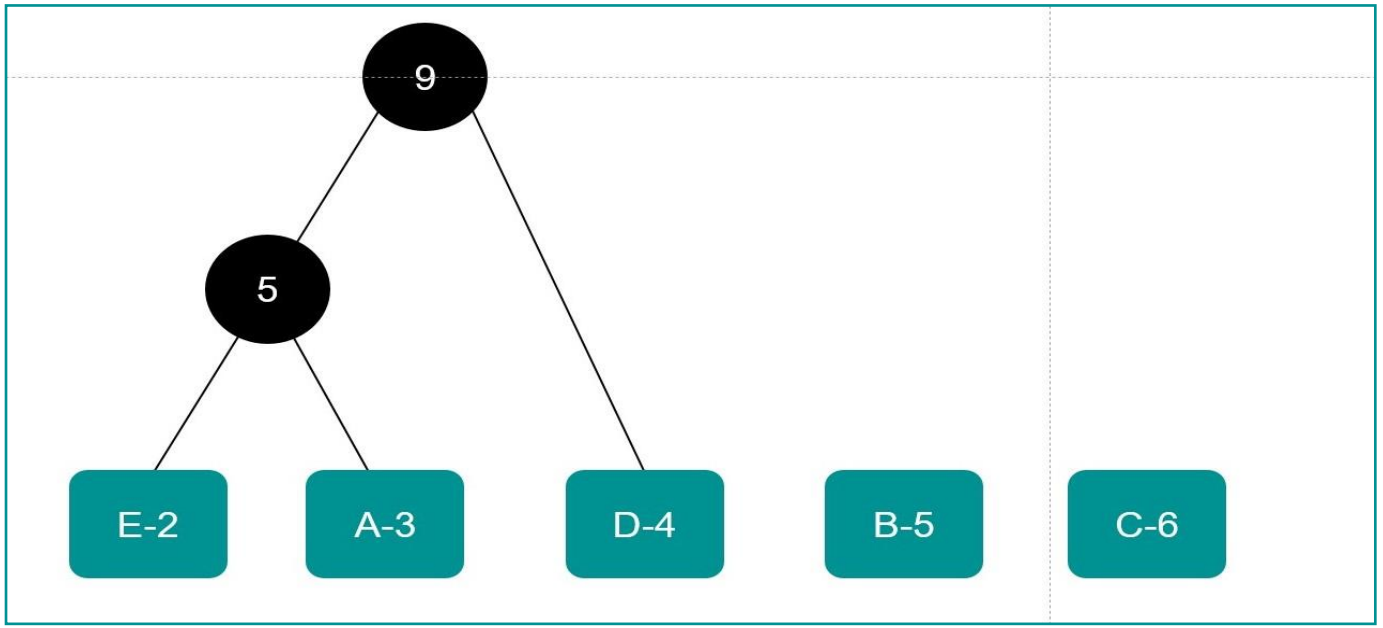


Letter	Count
A	3
B	5
C	6
D	4
E	2

Huffman codes Algorithm

Example .. **Message: (BCCABBDDAECCBBAEDDCC)**

Then between the root node 5 and counts take the minimum and add them up . Here 5 and 4 are minimum so we add them and make 9 as root node so continue the process.

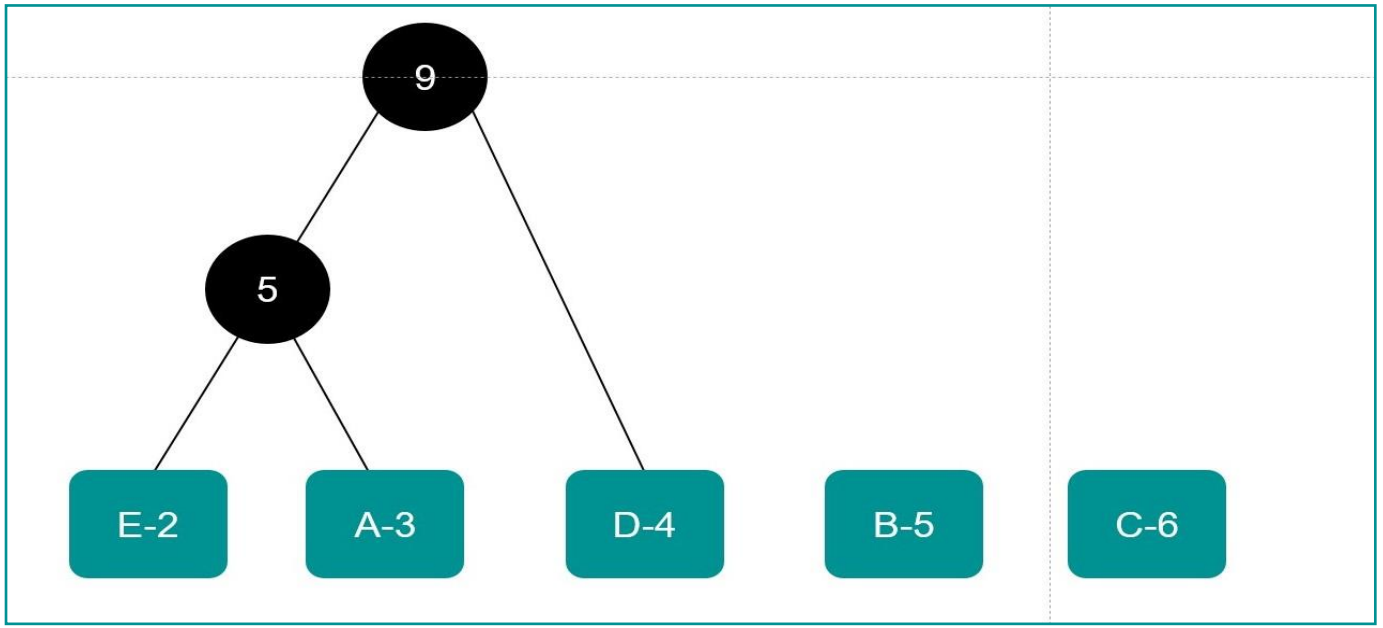


Letter	Count
A	3
B	5
C	6
D	4
E	2

Huffman codes Algorithm

Example .. **Message: (BCCABBDDAECCBBAEDDCC)**

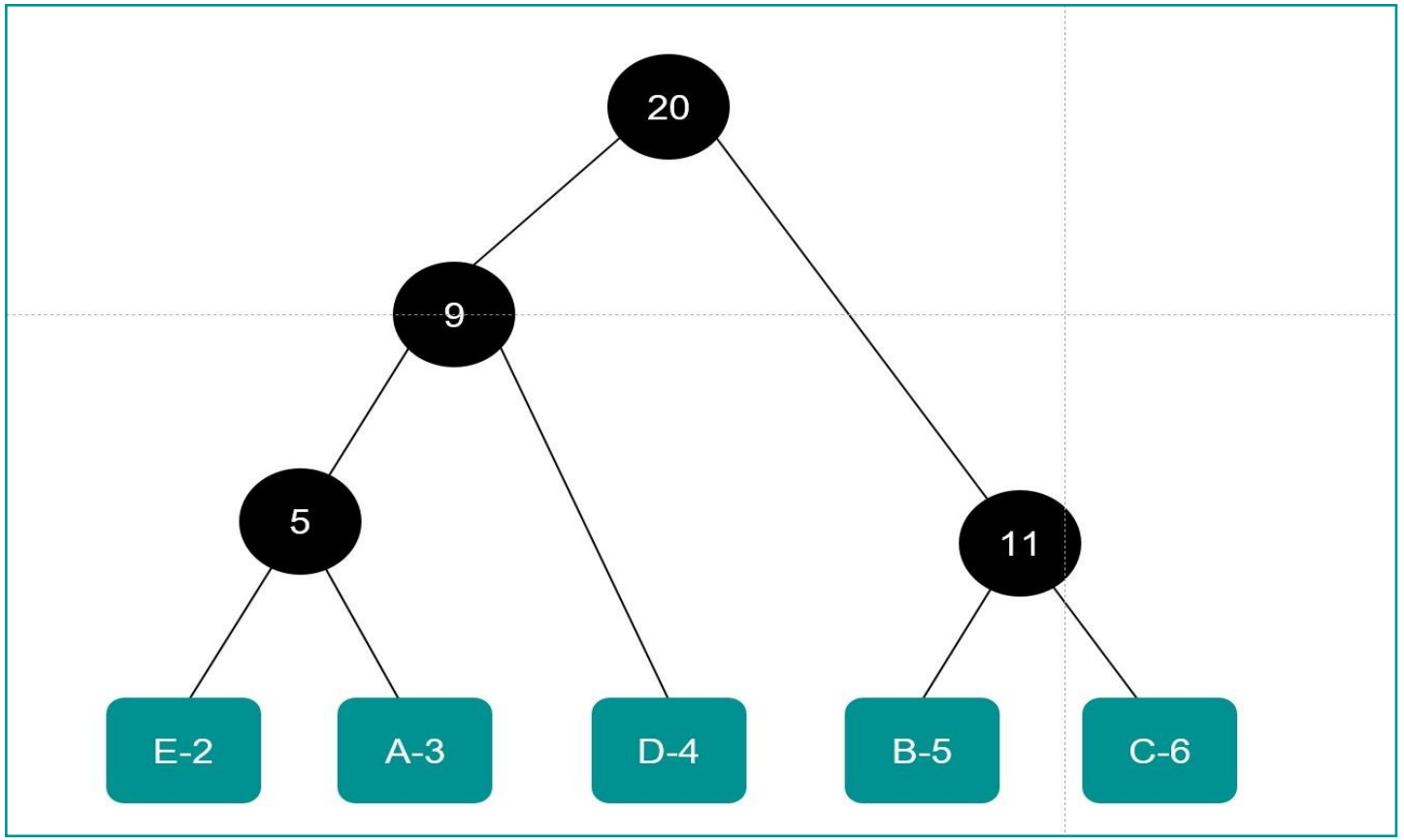
Then between the root node 5 and counts take the minimum and add them up . Here 5 and 4 are minimum so we add them and make 9 as root node so continue the process.



Letter	Count
A	3
B	5
C	6
D	4
E	2

Huffman codes Algorithm

Example .. **Message: (BCCABBDDAECCBBAEDDCC)**



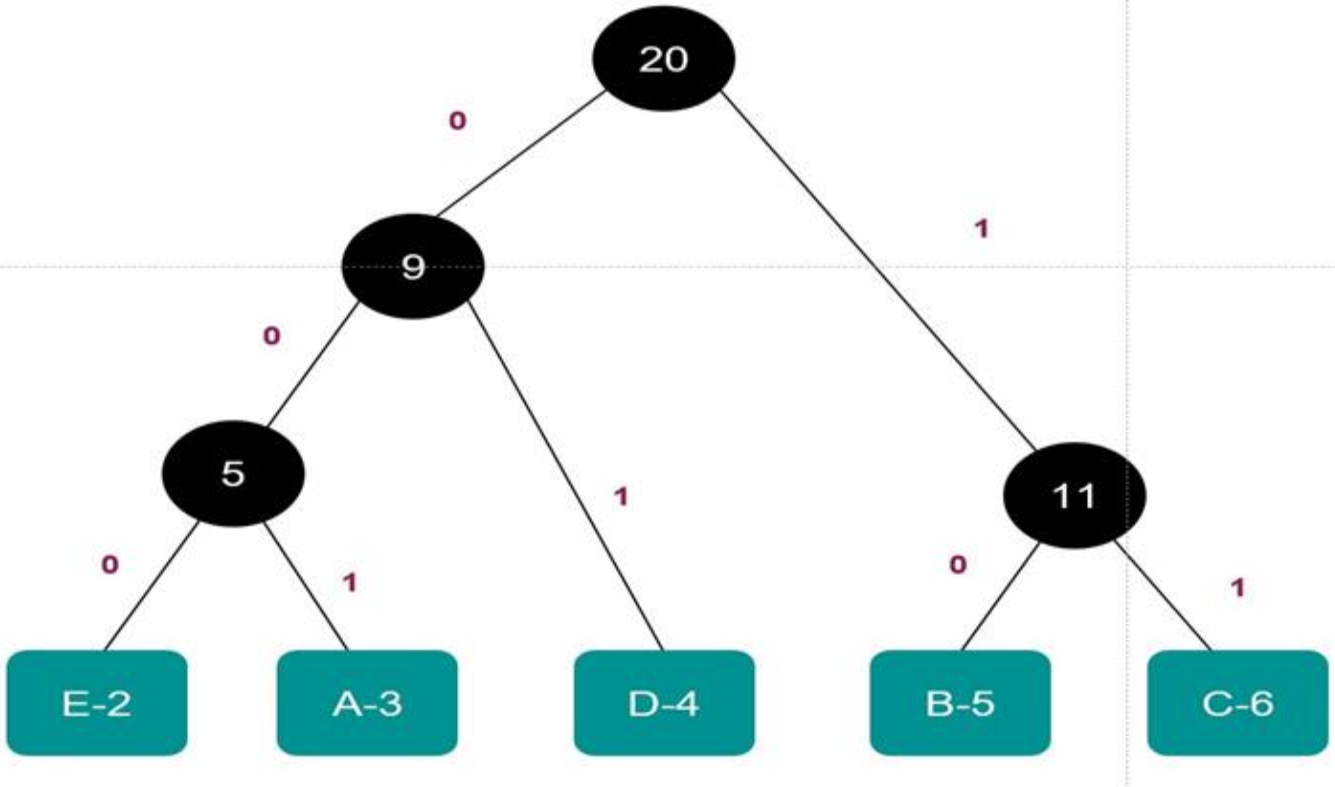
Letter	Count
A	3
B	5
C	6
D	4
E	2

Huffman codes Algorithm

Example : Message: (BCCABBDDAECCBBAEDDCC)

Mark the left hand edges as 0 and right hand edges as 1 and then traverse from root node to any letter . Suppose we want to go A from root node so the distance will be 001, for B 10 and so on.

Try to reduce the number of levels.



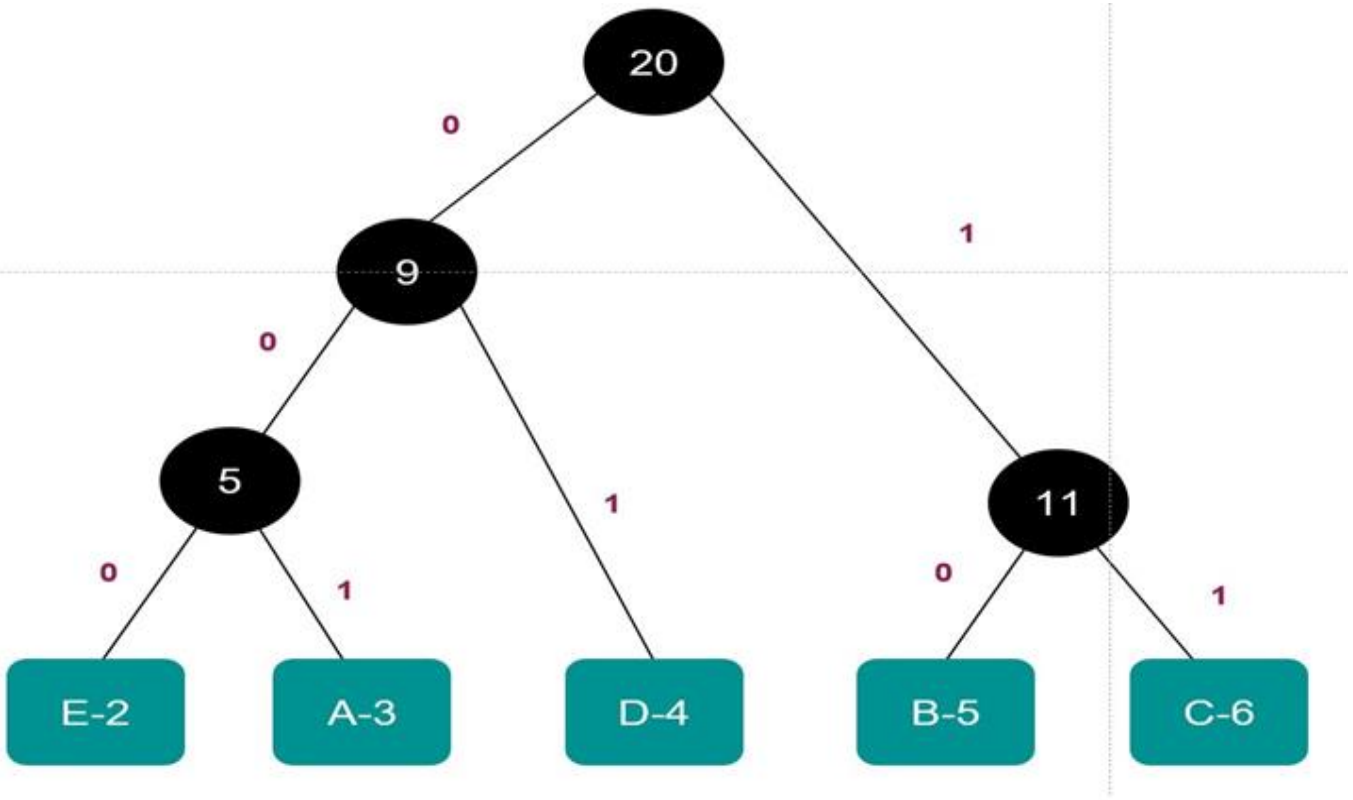
A	001
B	10
C	11
D	01
E	000

For A we need 3 bits ,B 2 bits , C 2 bits , D 2 bits, E 3 bits

For A, Count =3 So total bits 3*3=9 bits And so on

Huffman codes Algorithm

Example .. **Message: (BCCABBDDAECCBBAEDDCC)**



A	3	001	3×3=9
B	5	10	5×2=10
C	6	11	6×2=12
D	4	01	4×2=8
E	2	000	2×3=6

Total=45 bits

As we see first we do need (3x8+5x8+6x8+4x8+2x8=160) bits, and now we need 45 bits now we have compacted size the cost and size.

The End ..

This lecture was from

Introduction_to_Algorithms_Third_Edition_(2009)

chapter 16

